

netdev-0x14

Hanoch Haim Cisco Systems

TRex overview

The problem

- Carrier-Grade Traffic generators are
  - Expensive
  - Not scalable
  - Not flexible
- Implication
  - Limited and late testing
  - Different benchmarks and test methodologies
  - Real life bottlenecks and design issues undiscovered/untested

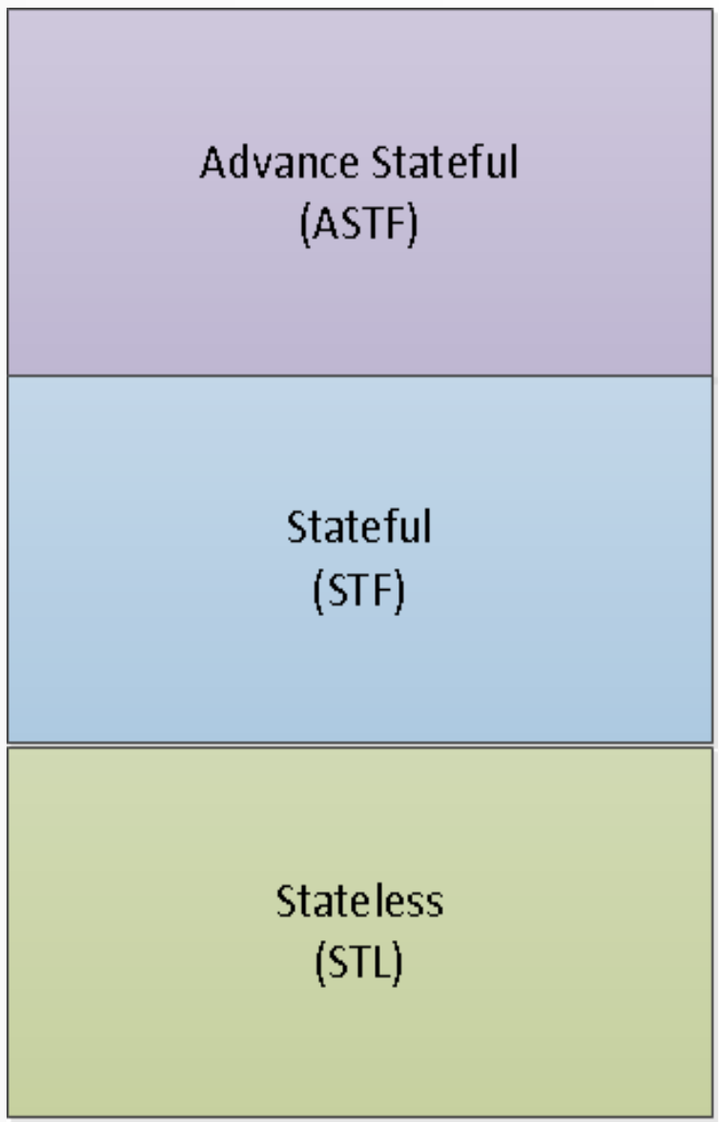
What is TRex?

- Linux user-space application
- Uses DPDK library
- Low cost: Standard server hardware and NICS
- High performance: up to 200 Gb/sec
- Flexible and Open Software
- Virtualization
- Easy installation and deployment

TRex modes of operation

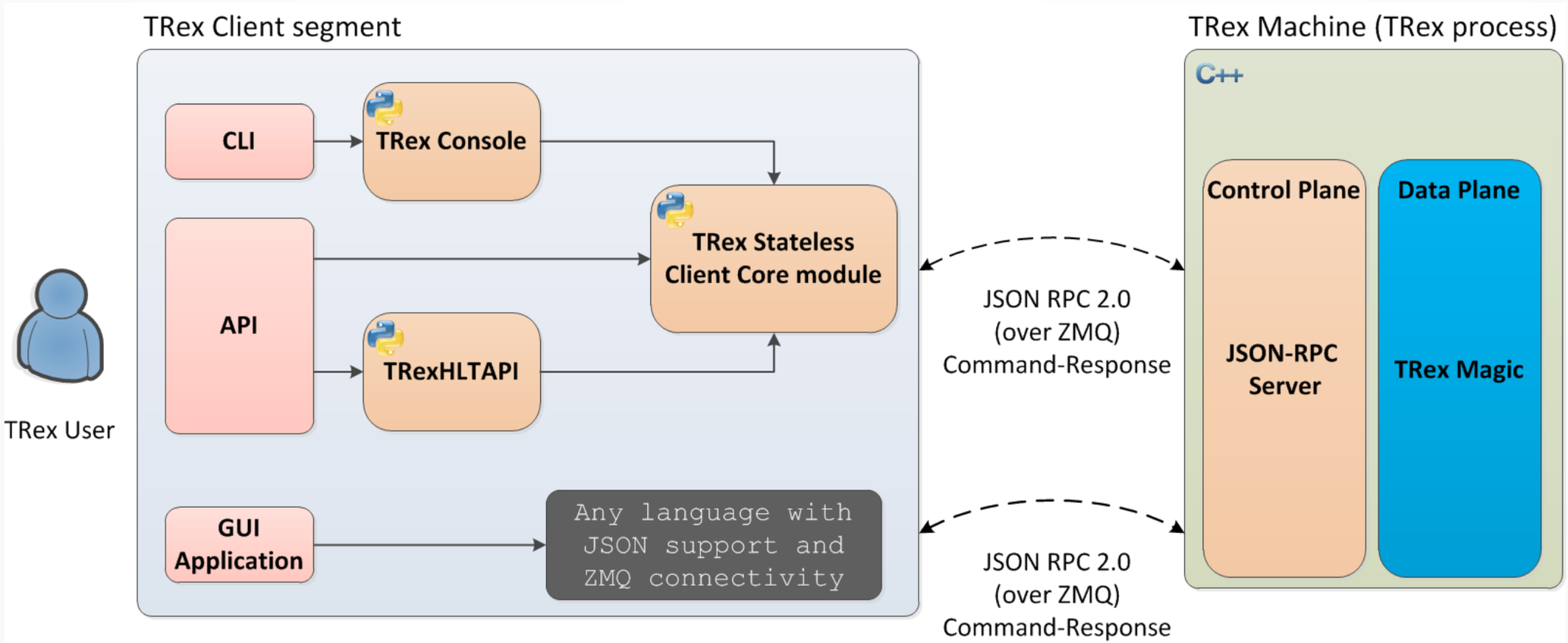
- Stateless for L2/L3
- Stateful for L7,i.e. DPI,NAT, Firewall
- BIRD for simulating routing protocols like BGP, OSPF, ISIS
- EMUfor client-side protocols simulation, i.e. Dot1x, dhcp, ipv6, ipv4

TRex main modes

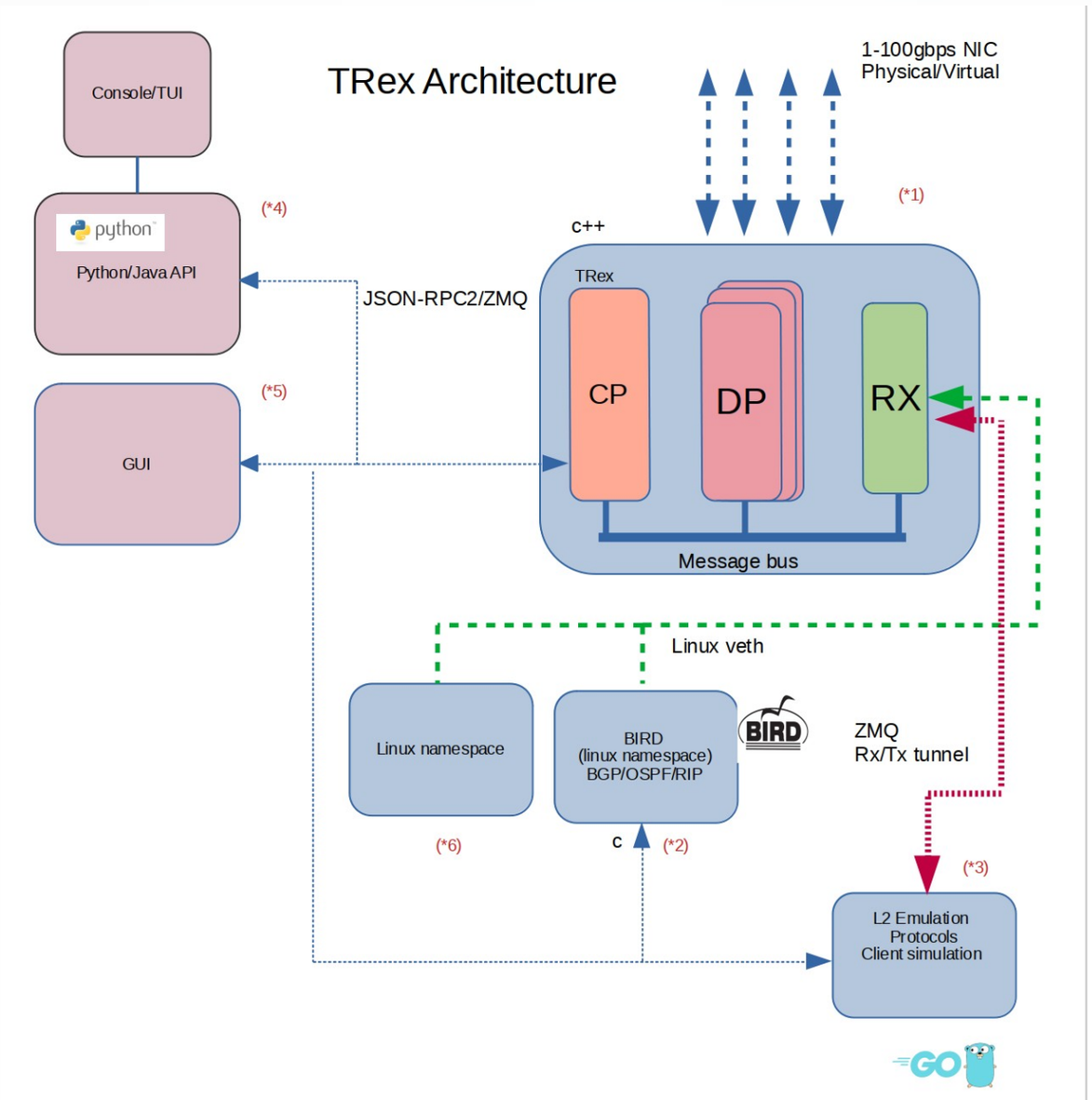


- L7, DUT terminate TCP/SSL, flow based
- DUT inspect L7. does not change TCP. Flow based
- DUT L2/L3 Switch , packet based

High level architecture



EMU/BIRD/LXE integration



Stateless

Stateless features

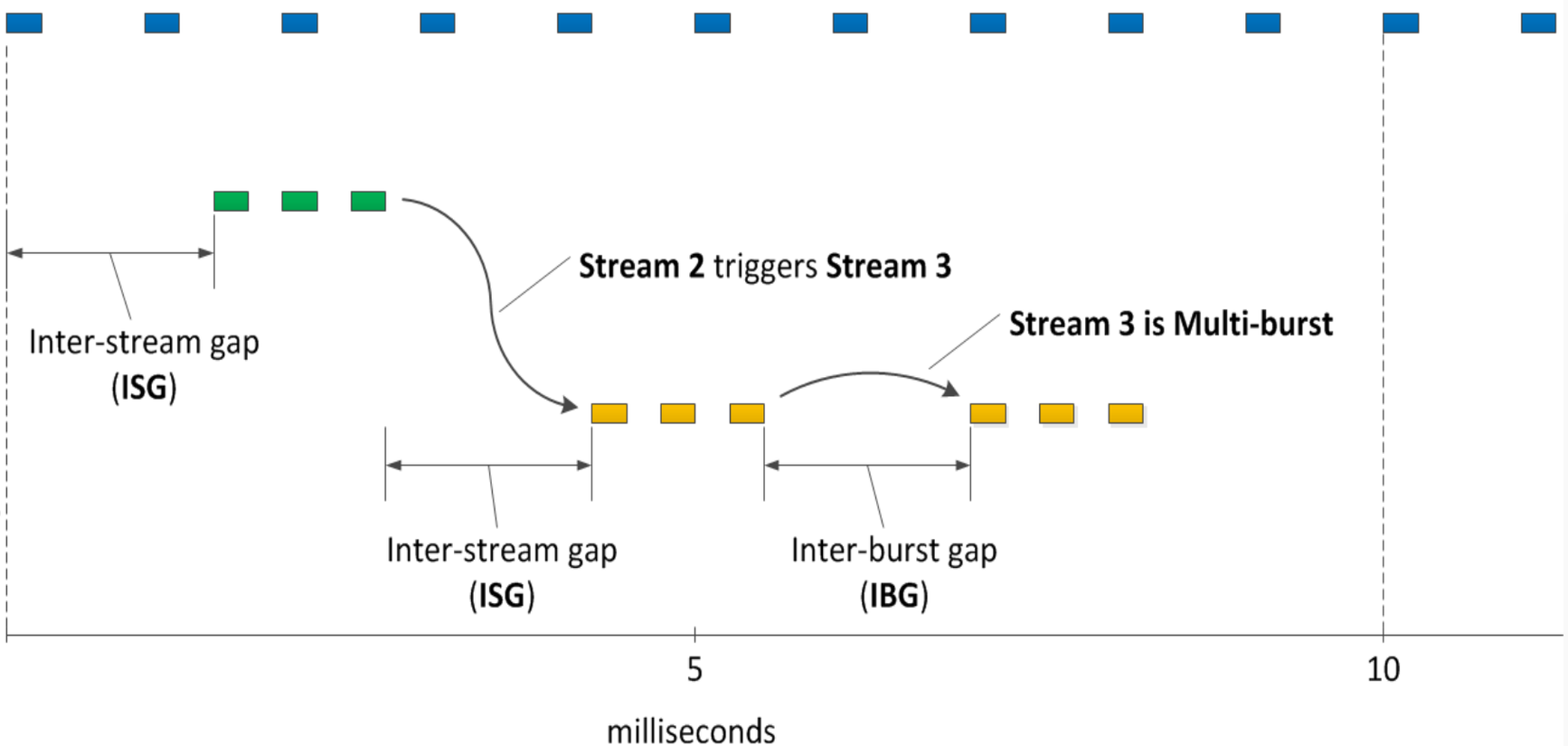
- High scale – ~10M-35MPPS/core
- Profile can support multiple streams, scalable to 20K parallel streams
- Interactive support – GUI/TUI
- Statistic per port/ stream (e.g. latency/jitter)
- Python automation support through a rich SDK
- Multi-user support
- Ability to capture to traffic

Example Profile

Stream 1
IP/TCP
Continuous 1 KPPS
10.0.0.1 – 10.0.0.255

Stream 2
IP/UDP
Burst 2 KPPS, 3 packets
16.0.0.1 – 10.0.0.255

Stream 3
MPLS/IP/GRE/VXLAN
Multi-burst 2 KPPS, 3 packets
200.0.0.1 – 200.0.0.255
Enabled by Stream 2



Python profile

```
def get_streams (self, direction = 0, **kwargs):
    # create 1 stream
    if direction==0:                                                    ❶
        src_ip="16.0.0.1"
        dst_ip="48.0.0.1"
    else:
        src_ip="48.0.0.1"
        dst_ip="16.0.0.1"

    pkt    = STLPktBuilder(
                          pkt = Ether()/IP(src=src_ip,dst=dst_ip)/
                          UDP(dport=12,sport=1025)/(10*'x') )

    return [ STLStream( packet = pkt,mode = STLTXCont()) ]
```

Multi burst

Stream 0

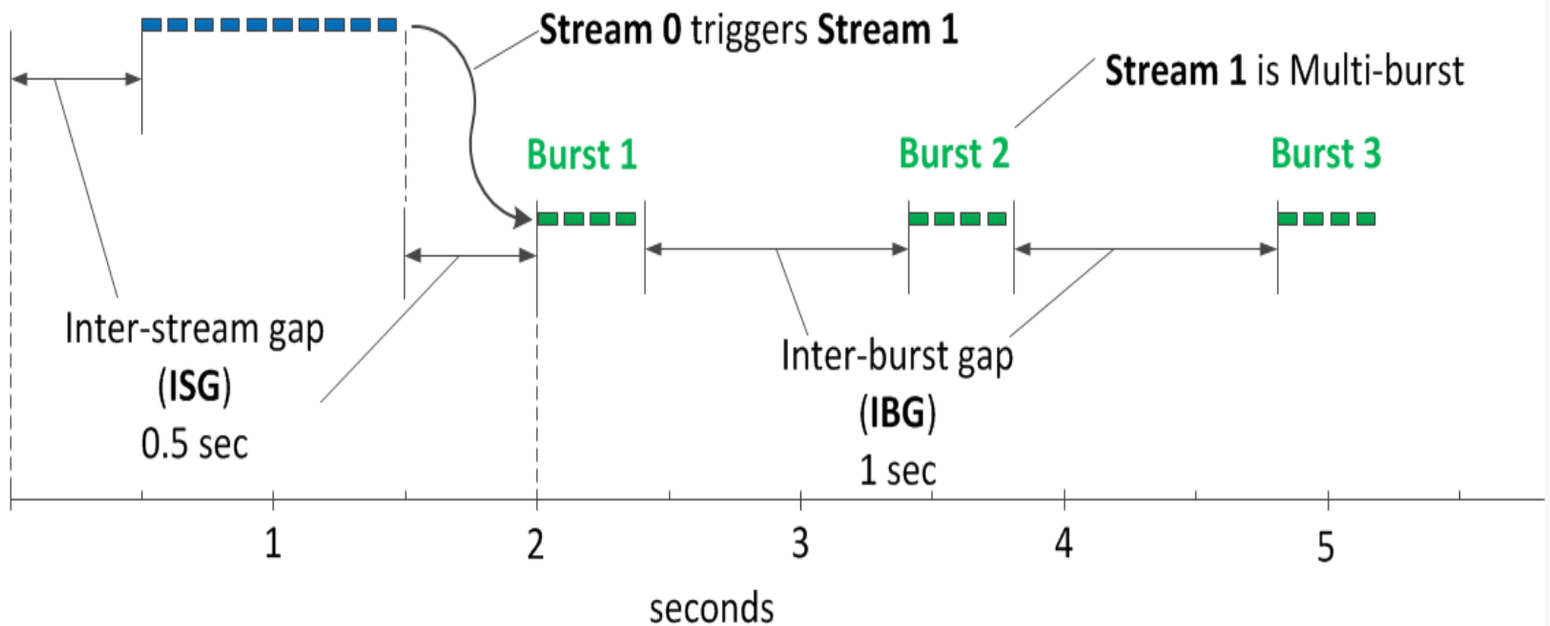
At start, inter-stream gap (**ISG**) of 0.5 sec

1 burst of 10 packets at 10 PPS

Stream 1

Triggered by Stream 0, inter-burst gap (**IBG**) of 1 sec

3 bursts of 4 packets at 10 PPS



Multi burst profile

```
    def create_stream (self):

        # create a base packet and pad it to size
        size = self.fsize - 4 # no FCS
        base_pkt =  Ether()/IP(src="16.0.0.1",dst="48.0.0.1")/UDP(dport=12,sport=1025)
        base_pkt1 =  Ether()/IP(src="16.0.0.2",dst="48.0.0.1")/UDP(dport=12,sport=1025)
        pad = max(0, size - len(base_pkt)) * 'x'


        return STLProfile( [ STLStream( isg = 10.0, # start in delay
❶                                       name     ='S0',
                                        packet = STLPktBuilder(pkt = base_pkt/pad),
                                        mode = STLTXSingleBurst( pps = 10, total_pkts = 10),
                                        next = 'S1'), # point to next stream

                             STLStream( self_start = False, # stream is disabled. Enabled by S0
❷                                       name     ='S1',
                                        packet   = STLPktBuilder(pkt = base_pkt1/pad),
                                        mode     = STLTXMultiBurst( pps = 1000,
                                                                    pkts_per_burst = 4,
                                                                    ibg = 1000000.0,
                                                                    count = 5)
                                        )

                            ]).get_streams()
```

Field Engine

Field Engine provides the ability to change any field inside a packet using a fast micro-code program:

- Change TOS
- Range of client IP: i.e. [10.0.0.1-10.0.0.254]
- Random packet size 64-9k
- Fields in tunnels i.e. GRE/MPLS/VxLAN/GTP

Field Engine - syn-attack

```
    def create_stream (self):

        # TCP SYN
        base_pkt  = Ether()/IP(dst="48.0.0.1")/TCP(dport=80,flags="S")        ❶


        # vm
        vm = STLScVmRaw( [ STLVmFlowVar(name="ip_src",
                                        min_value="16.0.0.0",
                                        max_value="18.0.0.254",
                                        size=4, op="random"),                 ❷

                           STLVmFlowVar(name="src_port",
                                        min_value=1025,
                                        max_value=65000,
                                        size=2, op="random"),                 ❸

                           STLVmWrFlowVar(fv_name="ip_src", pkt_offset= "IP.src" ), ❹

                           STLVmFixIpv4(offset = "IP"), # fix checksum        ❺

                           STLVmWrFlowVar(fv_name="src_port",                 ❻
                                          pkt_offset= "TCP.sport") # U

                         ]
                       )

        pkt = STLPktBuilder(pkt = base_pkt,
                            vm = vm)

        return STLStream(packet = pkt,
                         random_seed = 0x1234,# can be removed. will give the same random value
any run
                         mode = STLTXCont())
```

Automation

```
c = STLClient(username = "itay",server = "10.0.0.10", verbose_level = LoggerApi.VERBOSE_HIGH)

try:
    # connect to server
    c.connect()

    # prepare our ports (my machine has 0 <--> 1 with static route)
    c.reset(ports = [0, 1])

    # add both streams to ports
    c.add_streams(s1, ports = [0])

    # clear the stats before injecting
    c.clear_stats()

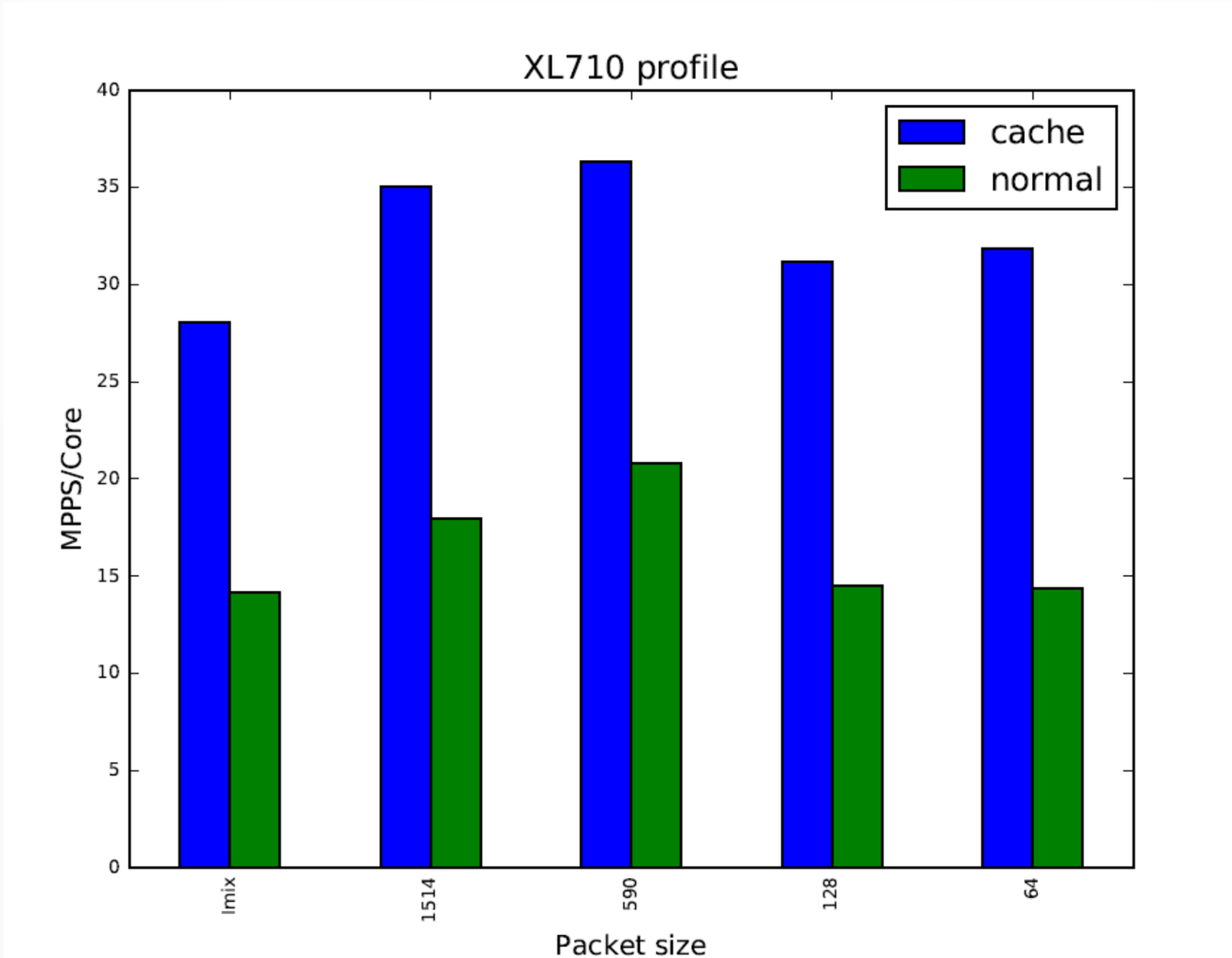
    c.start(ports = [0, 1], mult = "5mpps", duration = 10)

    # block until done
    c.wait_on_traffic(ports = [0, 1])

    # check for any warnings
    if c.get_warnings():
      # handle warnings here
      pass

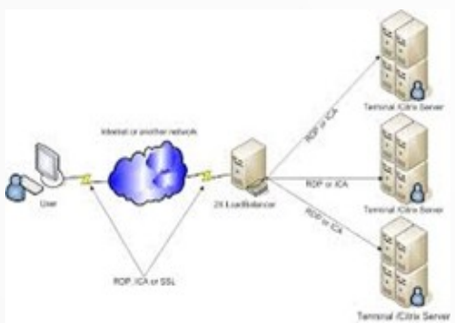
finally:
    c.disconnect()
```

MPPS/core

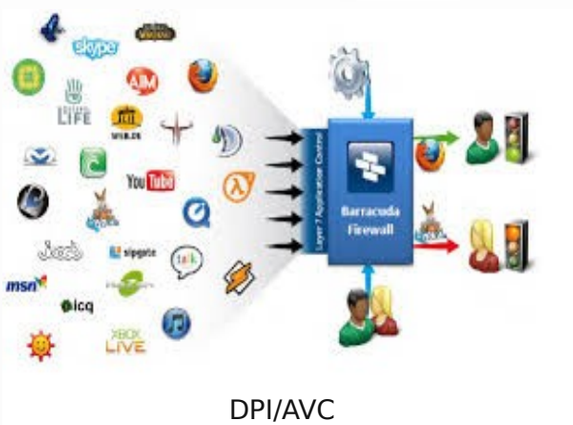

XL710 profile
cache
normal
MPPS/Core
0
5
10
15
20
25
30
35
40
Imix
1514
590
128
64
Packet size

Stateful

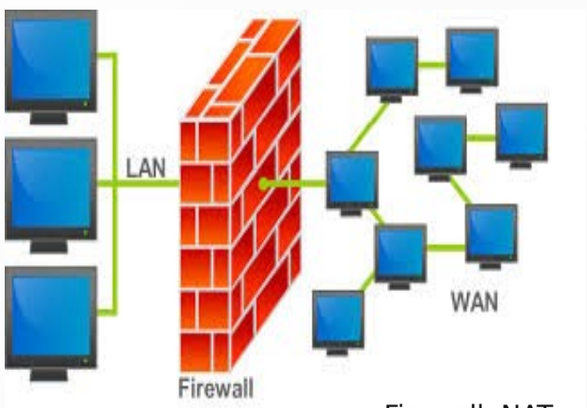
L7 Features



LB

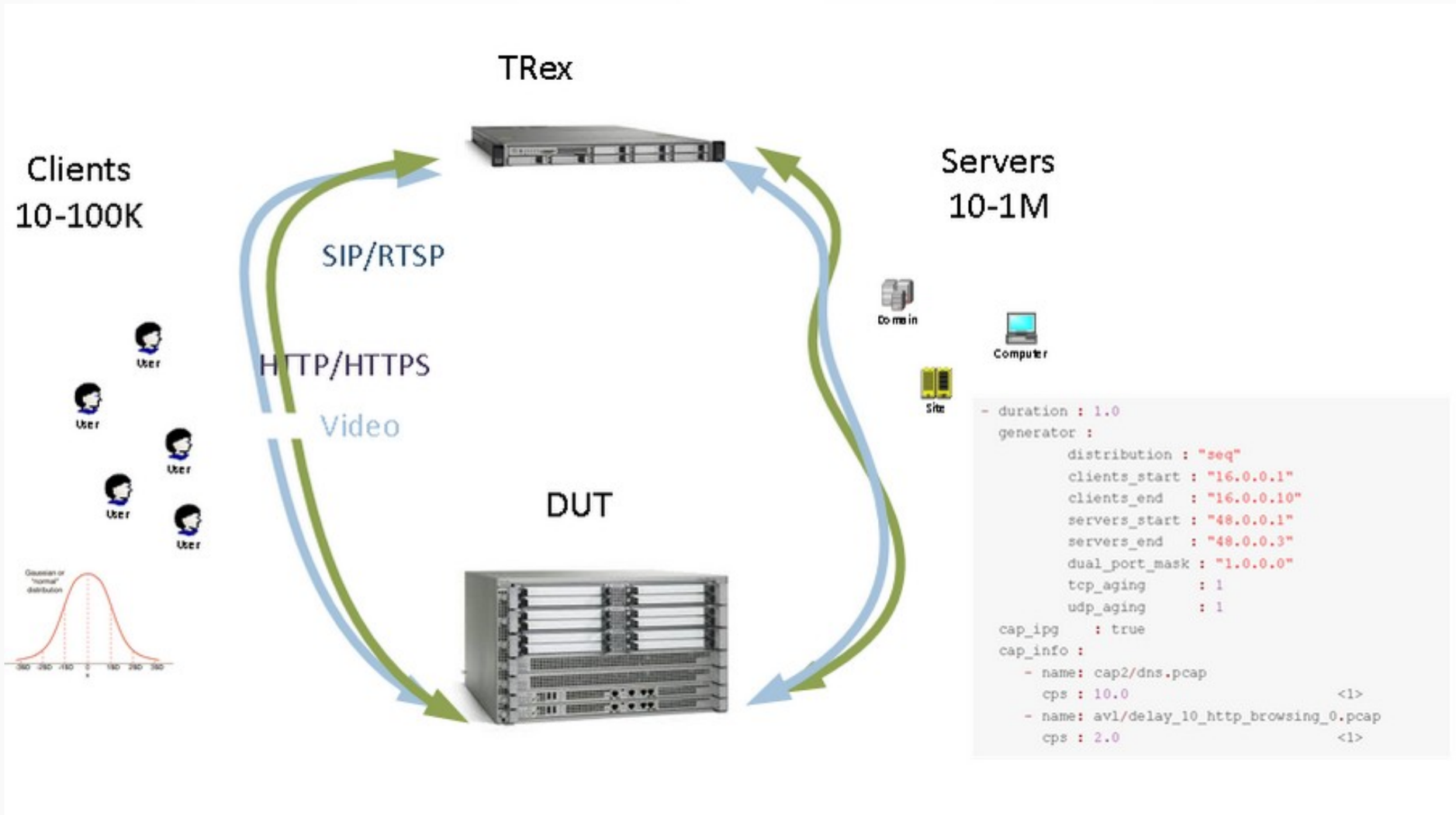


DPI/AVC

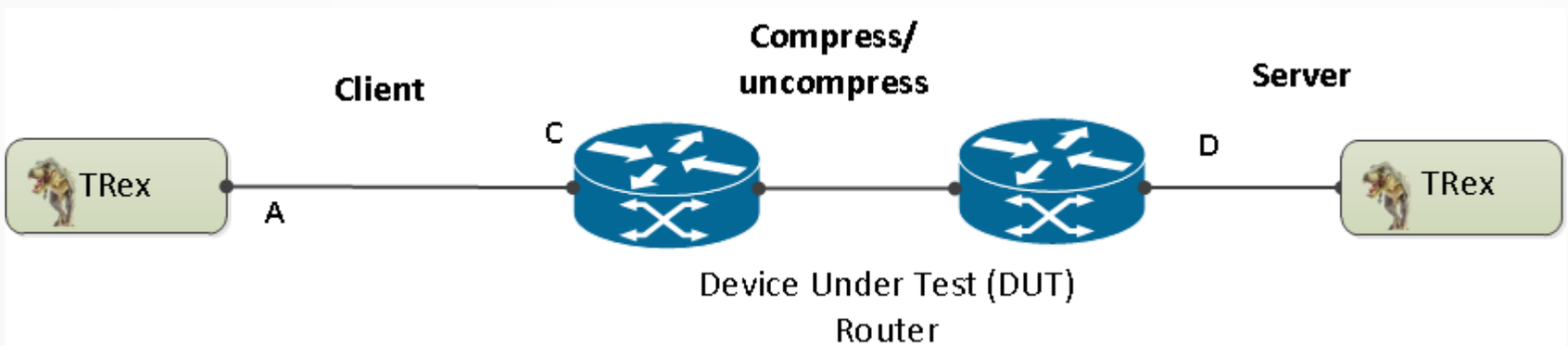


Firewall, NAT

Profile

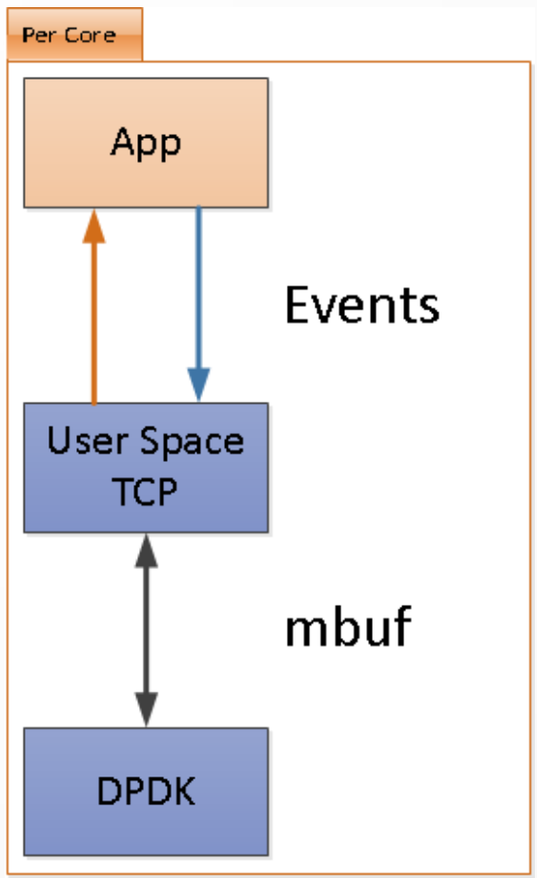

TRex
Clients
10-100K
Servers
10-1M
SIP/RTSP
HTTP/HTTPS
Video
DUT
User
Domain
Computer
Site
- duration : 1.0
generator :
distribution : "seq"
clients_start : "16.0.0.1"
clients_end : "16.0.0.10"
servers_start : "48.0.0.1"
servers_end : "48.0.0.3"
dual_port_mask : "1.0.0.0"
tcp_aging : 1
udp_aging : 1
cap_ipg : true
cap_info :
- name: cap2/dns.pcap
cps : 10.0 <1>
- name: avl/delay_10_http_browsing_0.pcap
cps : 2.0 <1>

Client/Server senario



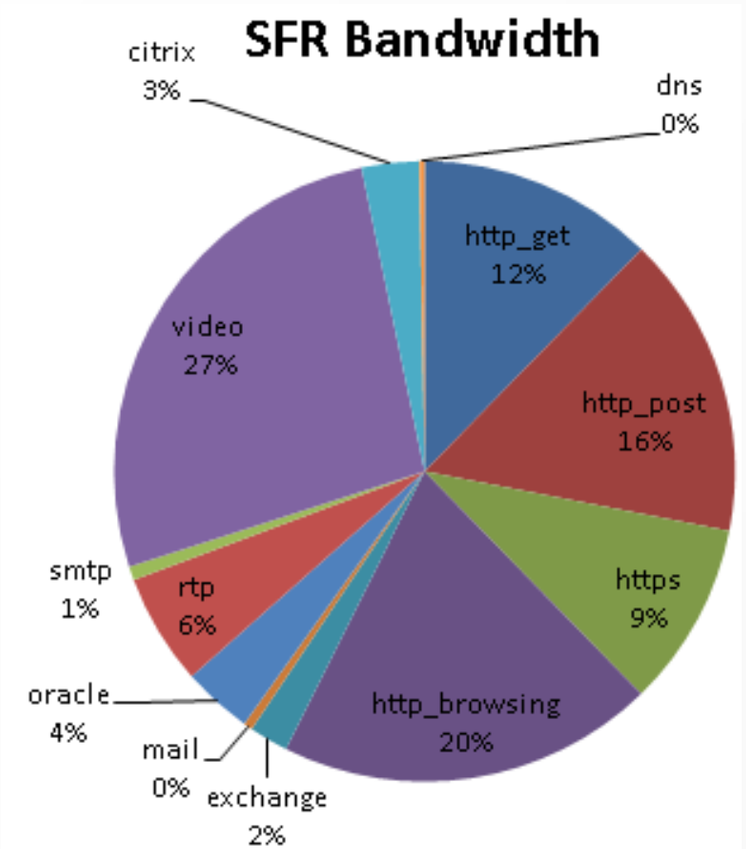
TCP stack

- BSD TCP stack
- High scale
- Interactive
- Accurate latency measurement
- Simple L7 emulation layer
- Emulate application using “micro-instructions”



EMIX profile

- HTTPS/RTP/HTTP/Video



A profile

```
from trex_astf_lib.api import *


class Prof1():
    def __init__(self):
        pass

    def get_profile(self):
        # ip generator
        ip_gen_c = ASTFIPGenDist(ip_range=["16.0.0.0", "16.0.0.255"],
                                 distribution="seq")
        ip_gen_s = ASTFIPGenDist(ip_range=["48.0.0.0", "48.0.255.255"],
                                 distribution="seq")
        ip_gen = ASTFIPGen(glob=ASTFIPGenGlobal(ip_offset="1.0.0.0"),      ❶
                           dist_client=ip_gen_c,
                           dist_server=ip_gen_s)

        return ASTFProfile(default_ip_gen=ip_gen,
                           cap_list=[ASTFCapInfo(
                                     file="../avl/delay_10_http_browsing_0.pcap"
                                     cps=1)
                                    ])                                     ❷


def register():
    return Prof1()
```

L7 Emulation layer

Client

```
# client commands
prog_c = ASTFProgram()
prog_c.send(http_req)
prog_c.recv(len(http_response))
```

Server

```
prog_s = ASTFProgram()
prog_s.recv(len(http_req))
prog_s.send(http_response)
```

Client side pseudo code

Client side pseudo code

```
template = choose_template()                                          ❶

src_ip,dest_ip,src_port = generate from pool of client
dst_port                = template.get_dest_port()

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)                 ❷

s.connect(dest_ip,dst_port)                                           ❸

# program                                                             ❹
s.write(template.request)       #write the following taken from the pcap file

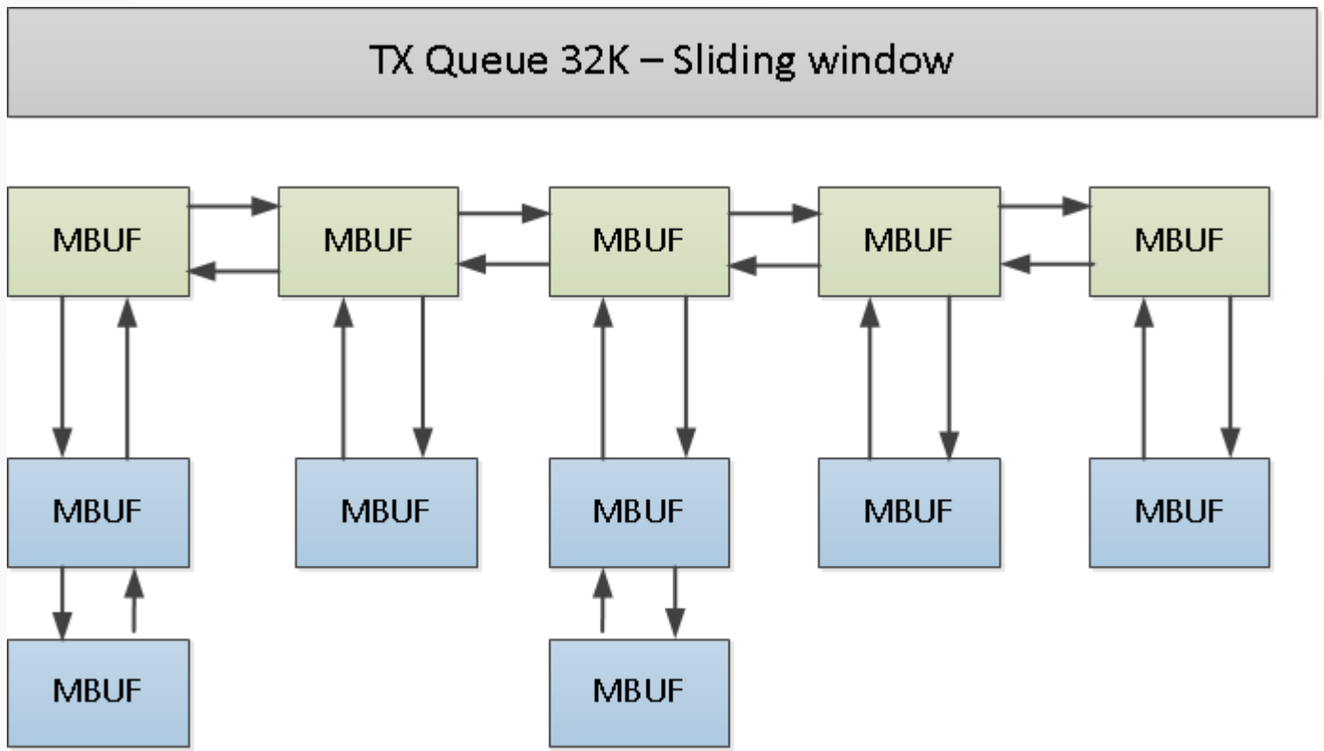
                                # GET /3384 HTTP/1.1
                                # Host: 22.0.0.3
                                # Connection: Keep-Alive
                                # User-Agent: Mozilla/4.0
                                # Accept: */*
                                # Accept-Language: en-us
                                # Accept-Encoding: gzip, deflate, compress


s.read(template.request_size)   # wait for 32K bytes and compare some of it

                                #HTTP/1.1 200 OK
                                #Server: Microsoft-IIS/6.0
                                #Content-Type: text/html
                                #Content-Length: 32000
                                # body ..


s.close();
```

TCP Stack scale



10M flows

320GByte

320M mbuf
=10GB

0.01GB

BIRD integration for routing protocols

Network simulation

BIRD can allow us to simulate routers by:

- Fast route injection
- Simulating adjacent routers
- Protocols like BGP, OSPF, RIP

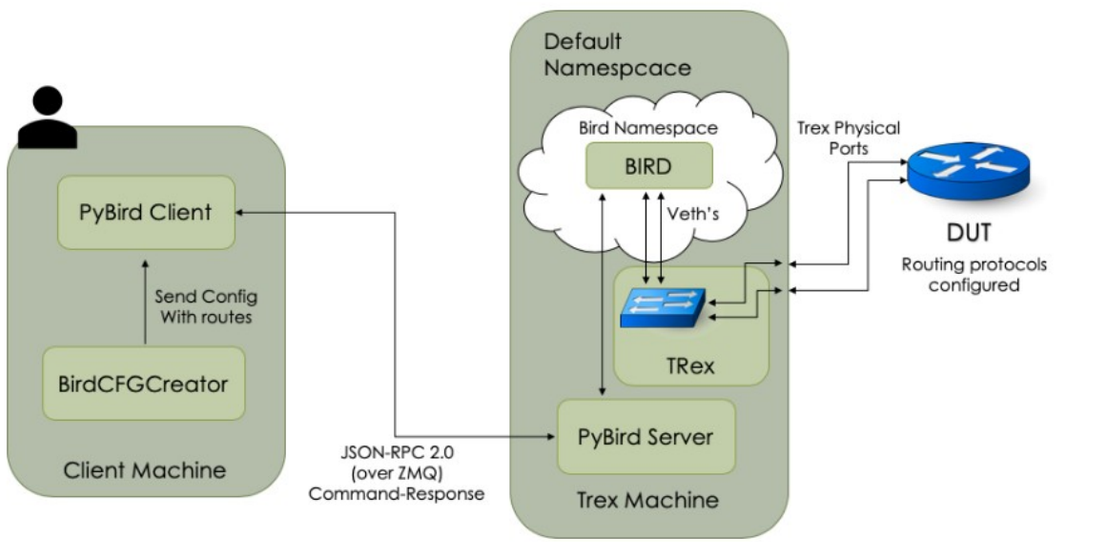


Figure 26. TRex-Bird General Scheme

Network simulation

```
> more bird/cfg/bgp.conf
router id 100.100.100.100;

protocol device {
    scan time 1;
}

protocol bgp my_bgp1 {
    local 1.1.1.3 as 65000;  # put your local ip and as number here
    neighbor 1.1.1.1 as 65000;  # put your dut ip and as number here
    ipv4 {
        import all;
        export all;
    };
}

# using a second interface
protocol bgp my_bgp2 {
    local 1.1.2.3 as 65000;  # same for the second interface
    neighbor 1.1.2.1 as 65000;
    ipv4 {
        import all;
        export all;
    };
}
```

```
asr1001-01#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
asr1001-01(config)#router bgp 65000
asr1001-01(config-router)#neighbor 1.1.1.3 remote-as 65000
asr1001-01(config-router)#neighbor 1.1.2.3 remote-as 65000
asr1001-01(config-router)#end
```

EMU service

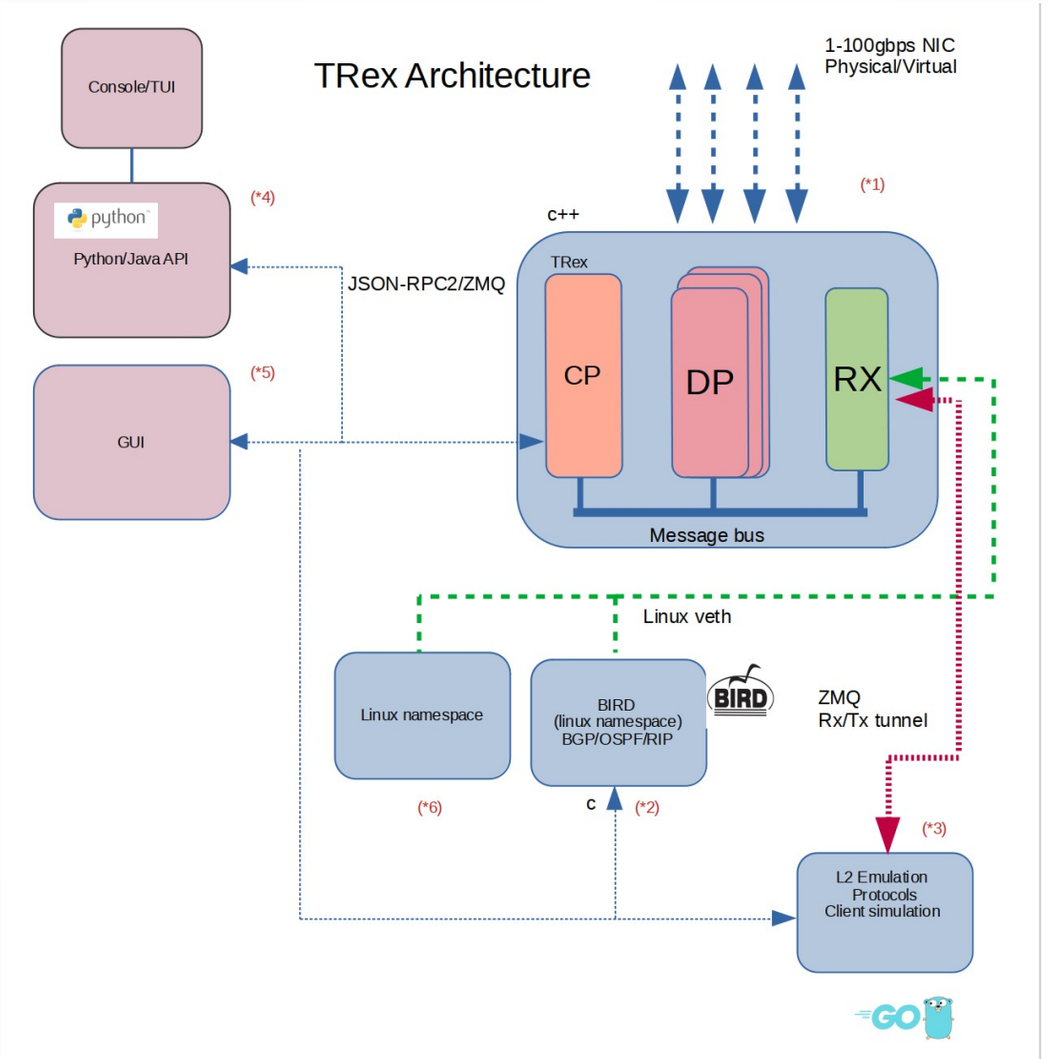
Why?

- L2/L3
  - IGMP/MLDv2 for simulation of video calls. DP/CP
  - Simulating IPv6/IPv4 protocols @ scale (~10k users)
- L7
  - Video calls (IGMP/MLDv2)
- Wireless
  - Clients are on the same network. DHCPv6/v4 and all the protocols to make the client alive
- Controler base (ISE/Radius)
  - 802.1x the port will be close without all the protocols/user-password

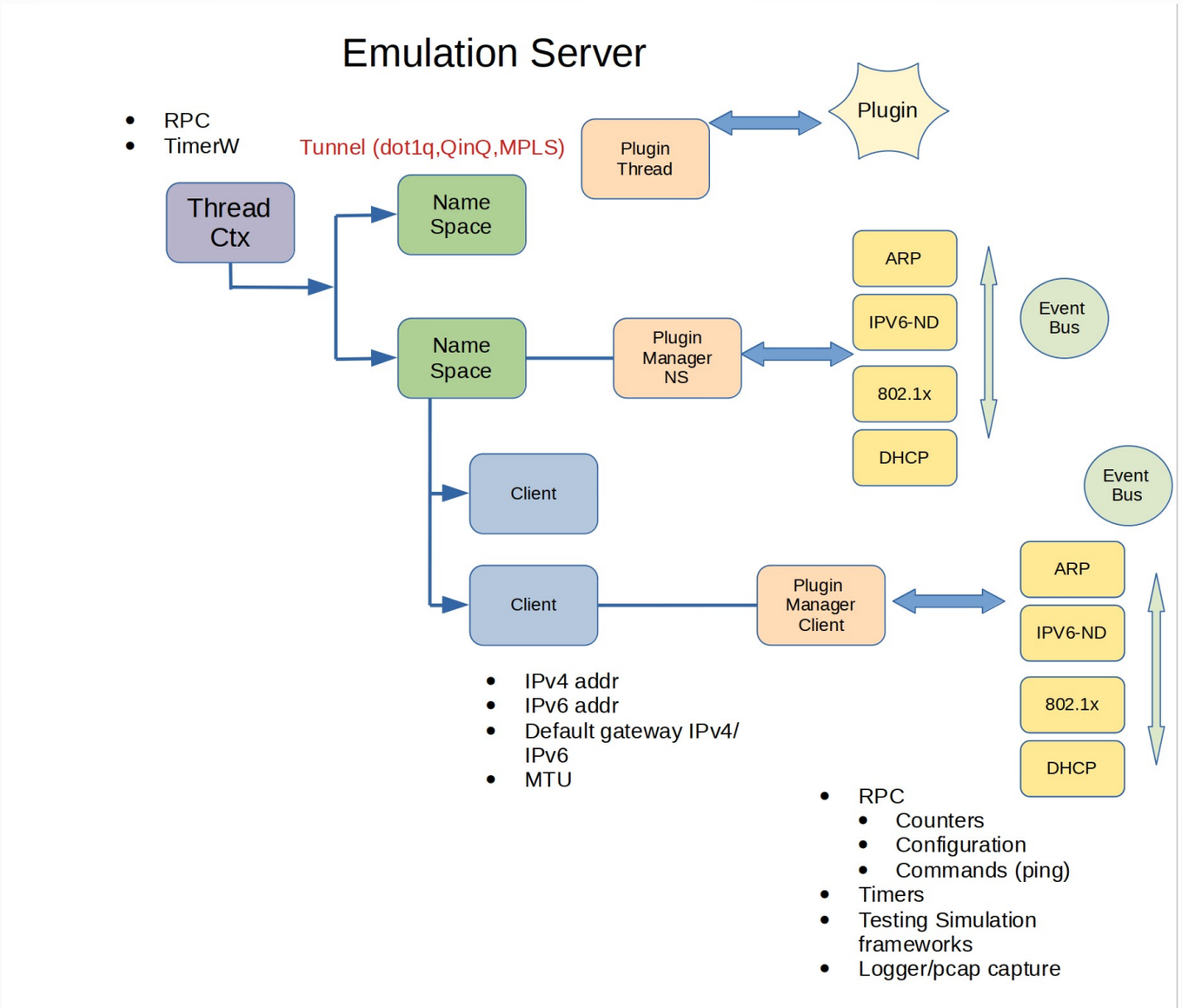
Client protocols example

- ARP – RFC 826
- ICMP – RFC 777
- IGMP – RFC 3376
- MLD – RFC 3810
- IPV6 - RFC 4443, RFC 486, RFC 4862
- DHCPv4 – RFC 2131
- DHCPv6 – RFC 8415
- 802.1x client side Radius
- Transport TCP/UDP/TLS for client auth
- Tunnels – VxLAN/GRE

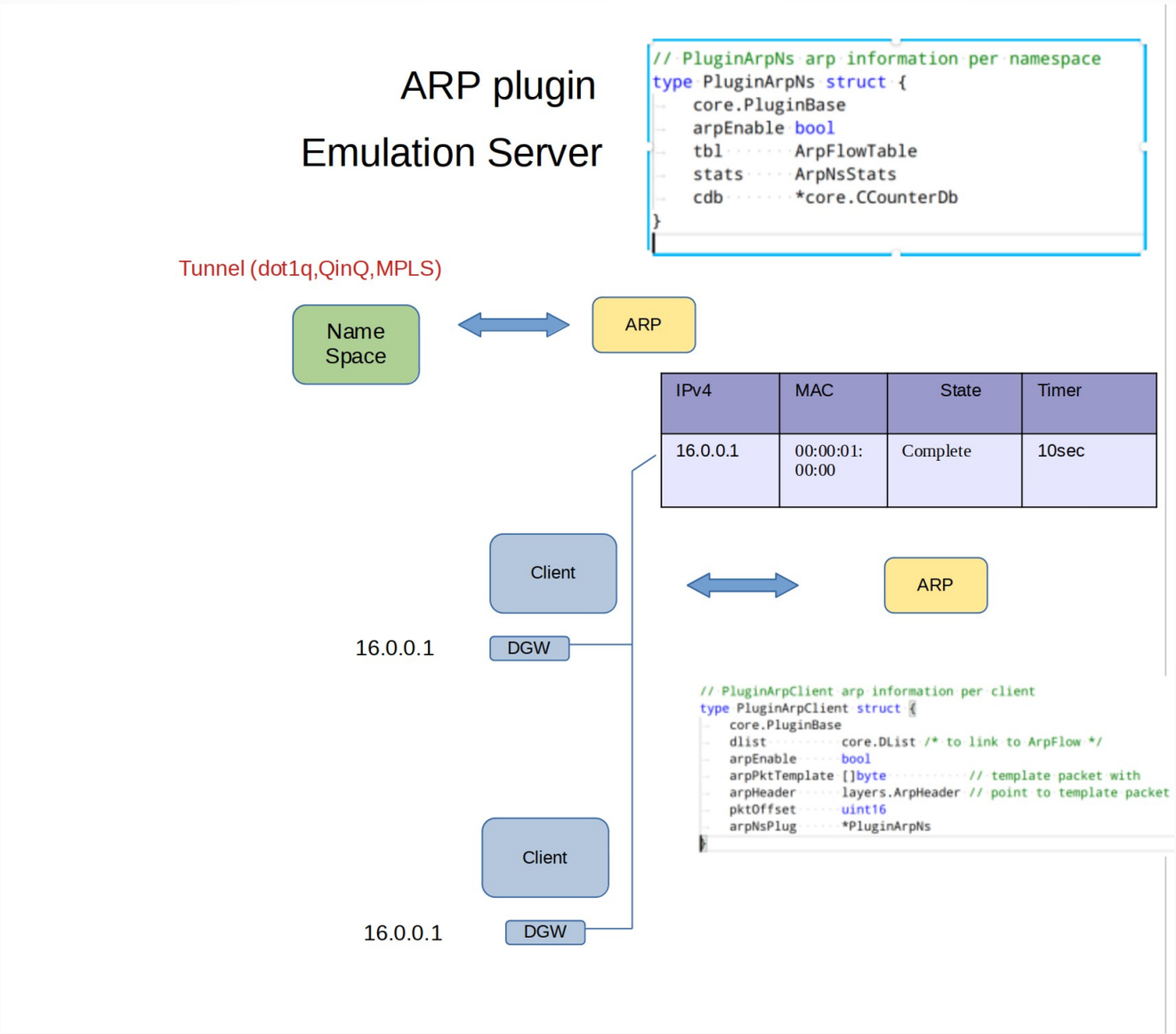
EMU/BIRD/LXE


TRex Architecture
Console/TUI
python
Python/Java API
GUI
(*4)
(*5)
JSON-RPC2/ZMQ
1-100gbps NIC
Physical/Virtual
(*1)
c++
TRex
CP
DP
RX
Message bus
Linux veth
Linux namespace
BIRD
(linux namespace)
BGP/OSPF/RIP
BIRD
ZMQ
Rx/Tx tunnel
(*6)
c
(*2)
(*3)
L2 Emulation
Protocols
Client simulation
GO

Emulation server



ARP plugin


ARP plugin
Emulation Server
// PluginArpNs arp information per namespace
type PluginArpNs struct {
core.PluginBase
arpEnable bool
tbl ArpFlowTable
stats ArpNsStats
cdb *core.CCounterDb
}
Tunnel (dot1q,QinQ,MPLS)
Name Space
ARP
IPv4
MAC
State
Timer
16.0.0.1
00:00:01:00:00
Complete
10sec
Client
ARP
16.0.0.1
DGW
// PluginArpClient arp information per client
type PluginArpClient struct {
core.PluginBase
dlist core.DList /* to link to ArpFlow */
arpEnable bool
arpPktTemplate []byte // template packet with
arpHeader layers.ArpHeader // point to template packet
pktOffset uint16
arpNsPlug *PluginArpNs
}
Client
16.0.0.1
DGW

Profile example

```
class Prof1():
    def __init__(self):
        self.def_ns_plugs  = None
        self.def_c_plugs  = None

    def create_profile(self, ns_size, clients_size):
        ns_list = []

        # create different namespace each time
        vport, tci, tpid = 0, [0, 0], [0x00, 0x00]
        for j in range(vport, ns_size + vport):
            ns_key = EMUNamespaceKey(vport  = j,
                                     tci    = tci,
                                     tpid   = tpid)
            ns = EMUNamespaceObj(ns_key = ns_key, def_c_plugs = self.def_c_plugs)

            mac = Mac('00:00:00:70:00:01')
            ipv4 = Ipv4('1.1.5.2')
            dg = Ipv4('1.1.5.1')
            u = "test{}".format(i+1)

            # create a different client each time
            for i in range(clients_size):
                client = EMUClientObj(mac     = mac[i].V(),
                                      ipv4    = ipv4[i].V(),
                                      ipv4_dg = dg.V(),
                                      plugs   = {'arp': {},
                                                 'icmp': {},
                                                 'dot1x': {'user':u, 'password':u, 'flags':1},
                                                },
                                      )
                ns.add_clients(client)
            ns_list.append(ns)

        return EMUProfile(ns = ns_list, def_ns_plugs = self.def_ns_plugs)
```